Hooking a native function scooped within a function we are interested in :

```
1
2  bool Java_com_mobilehackinglab_FridaEight_MainActivity_strCmp
3                 (_JNIEnv *param_1,undefined8 param_2,_jstring *param_3)
4
5  {
6    int iVar1;
7    char *__s1;
8    ulong uVar2;
9    long lVar3;
10   int local_cc;
11   char acStack_7c [100];
12   long local_18;
13
14   lVar3 = tpidr_el0;
15   local_18 = *(long *)(lVar3 + 0x28);
16   __s1 = (char *)_JNIEnv::GetStringUTFChars(param_1,param_3,(uchar *)0x0);
17   local_cc = 0;
18   while( true ) {
19     uVar2 = __strlen_chk("NIM|JOUSP`UP`OBUJWFMBOE~",0xffffffffffffffff);
20     if (uVar2 <= (ulong)(long)local_cc) break;
21     acStack_7c[local_cc] = "NIM|JOUSP`UP`OBUJWFMBOE~"[local_cc] + -1;
22     local_cc = local_cc + 1;
23   }
24   lVar3 = __strlen_chk("NIM|JOUSP`UP`OBUJWFMBOE~",0xffffffffffffffff);
25   acStack_7c[lVar3] = '\0';
26   iVar1 = strcmp(__s1,acStack_7c);
27   _JNIEnv::ReleaseStringUTFChars(param_1,param_3,__s1);
28   lVar3 = tpidr_el0;
29   lVar3 = *(long *)(lVar3 + 0x28) - local_18;
30   if (lVar3 == 0) {
31     return iVar1 == 0;
32   }
33                     /* WARNING: Subroutine does not return */
```

```
var moduleName = "libFridaEight.so";

var functionName = "Java_com_mobilehackinglab_FridaEight_MainActivity_strCmp";



function waitForModule(modName, callback) {

    var mod = Process.findModuleByName(modName);

    if (mod !== null) {

        callback(mod);
```

```
        return;
    }
    setTimeout(function () {
        waitForModule(modName, callback);
    }, 50);
}
```

This part , we are getting the offset and address of the libFridaEight.so module which contains the target function address
But it solves a problem when working with native functions , it waits for the module to get loaded first so it can hook its address , the issue can be faced when we spawn the target application using the script instead of hooking running one
This function just checks every 50ms until the module shows up, then calls `callback(mod)` with the resolved module object. It's a simple recursive poll

```
waitForModule(moduleName, function (mod) {
    console.log("[+] " + moduleName + " loaded @ " + mod.base);
```

Now we know the module is loaded and where its base address is

```
   var targetFuncAddr = mod.findExportByName(functionName);
   console.log("[*] Target function " + functionName + " @ " +
targetFuncAddr);


    var libc = Process.findModuleByName("libc.so");
    var strcmpAddr = libc.findExportByName("strcmp");
    console.log("[*] strcmp @ " + strcmpAddr);
```

Then normally hook the target function address
hook every strcmp function call , but the issue is , it will hook every call even the ones we are not interested in

```
  Interceptor.attach(strcmpAddr, {

          onEnter: function (args) {

              var callerAddr = this.returnAddress;

              var offset = callerAddr.sub(targetFuncAddr).toInt32();



              if (offset >= 0 && offset < 0x120) {

                  var a = args[0].readCString();

                  var b = args[1].readCString();

                  console.log("[*] strcmp() inside strCmp - arg1: " + a + " |
arg2: " + b);

              }

          }

      });
```

And here is the solution :
get the `this.returnAdress` of the strcmp , which is the exact address the execution flow was at before jumping to execute the strcmp function (where the context it was called from was)

Then get the offset before the target function
Check if the strcmpAddr was within the range of the offset-targetfunction and offset+targetfunction

if true , this means the function was called inside the target function's range then this is the call we are interested in , print its args